

AI-POWERED RESUME SCREENING & INTELLIGENT JOB MATCHING SYSTEM

Detailed Project Documentation

Project Type: AI/ML + Natural Language Processing Application

Technology: Python, Scikit-learn, NumPy, NLP, TF-IDF, Cosine Similarity, Streamlit

Purpose: Automated resume-to-job-description matching and candidate ranking

Prepared for: GitHub, LinkedIn Portfolio, Technical Interviews and Project Presentation

Table of Contents

No.	Section
1	Executive Summary
2	Problem Statement
3	Objectives
4	Scope
5	Functional Requirements
6	Non-Functional Requirements
7	System Architecture
8	End-to-End Workflow
9	Project Structure
10	Module Description
11	NLP & Matching Methodology
12	TF-IDF
13	Cosine Similarity
14	Scoring & Ranking
15	Streamlit Application
16	Installation & Setup
17	How to Run
18	Testing Strategy
19	Troubleshooting
20	Security & Privacy
21	Limitations
22	Future Enhancements
23	Deployment
24	Conclusion
25	GitHub README Guidance

1. Executive Summary

The AI-Powered Resume Screening & Intelligent Job Matching System is an end-to-end NLP/ML application that automates the initial comparison of resumes with job descriptions.

The system accepts resume content and a target job description, extracts and preprocesses text, converts documents into numerical representations using TF-IDF, calculates cosine similarity, generates relevance scores, and ranks candidates.

The application is implemented in Python using Scikit-learn and is presented through Streamlit. Its modular design provides a foundation for future semantic embeddings, transformer models, LLM-based extraction, RAG and vector search.

2. Problem Statement

Recruiters may need to review a large number of resumes for a single position. Manual comparison is time-consuming and can become inconsistent as application volume increases.

The project addresses this by providing a machine-assisted relevance signal that helps identify resumes whose textual content is more closely aligned with a job description. It is a decision-support tool and should not be treated as an autonomous hiring decision system.

3. Objectives

Automate initial resume screening; process unstructured resume text; extract useful textual features; compare resumes with job descriptions; calculate relevance scores; rank candidates; provide an easy-to-use interface; and maintain a modular codebase suitable for future AI/ML upgrades.

4. Scope of the System

Input: Resume documents/text and a target job description.

Processing: Text extraction, normalization/preprocessing, vectorization and similarity calculation.

Output: Candidate matching/relevance score and ranking.

Interface: Interactive Streamlit web application.

Extension: Architecture can be upgraded with semantic embeddings, LLMs, RAG, vector databases and explainable matching.

5. Functional Requirements

Resume upload/input; job-description input; document text extraction; common preprocessing; TF-IDF vectorization; cosine similarity; score generation; candidate ranking; input validation; and interactive result display.

6. Non-Functional Requirements

Usability: Simple interface for technical and non-technical users.

Maintainability: Separate UI, parsing, preprocessing and matching responsibilities.

Performance: Lightweight TF-IDF/cosine operations for typical resume collections.

Reliability: Reproducible dependency environment and input validation.

Scalability: Matching layer should be replaceable with embedding/vector-search methods.

Privacy: Resume information must be handled securely and only stored when necessary.

7. System Architecture

High-Level Architecture

```
Resume Upload/Input
↓
Document/Text Extraction
↓
NLP Preprocessing
↓
TF-IDF Vectorization
↓
Cosine Similarity / Matching Engine
↓
Score Calculation
↓
Candidate Ranking
↓
Streamlit Dashboard
```

The architecture separates the presentation layer from the core matching logic. This makes individual components easier to test and allows the matching algorithm to be replaced without rebuilding the complete interface.

8. End-to-End Workflow

1. **Resume Input:** The user uploads or provides candidate resumes.
2. **Job Description:** The target role requirements become the reference text.
3. **Text Extraction:** Supported document content is converted into machine-readable text.
4. **Preprocessing:** Text is normalized and cleaned consistently for both resumes and job descriptions.
5. **Feature Engineering:** TF-IDF converts documents into sparse numerical vectors.
6. **Similarity:** Cosine similarity compares each resume vector with the job-description vector.
7. **Scoring:** Similarity values are presented as relevance scores.
8. **Ranking:** Candidates are sorted from higher to lower relevance.
9. **Presentation:** Streamlit displays the results interactively.

9. Project Structure

```
resume_screener/
  app.py
  requirements.txt
  README.md
  utils/
    matcher.py
    parser.py
    preprocessing.py
```

app.py: Streamlit entry point and UI orchestration.

matcher.py: Core matching/scoring logic and TF-IDF functionality.

parser.py: Document-to-text extraction where applicable.

preprocessing.py: Text normalization/cleaning where separated into a module.

requirements.txt: Reproducible Python dependencies.

README.md: GitHub-facing project documentation.

10. Module Description

Presentation Layer — app.py: Handles Streamlit widgets, input collection, calls to backend functions and result display. Keeping heavy processing outside the UI improves maintainability.

Matching Layer — matcher.py: Contains the NLP matching workflow. The current implementation uses Scikit-learn's TfidfVectorizer and similarity-based scoring.

Parsing Layer: Converts supported resume files into text while attempting to preserve meaningful content such as skills, technologies, qualifications and experience.

Preprocessing Layer: Applies consistent cleaning/normalization rules to both sides of the comparison.

11. NLP & Matching Methodology

The project uses a classical information-retrieval approach. It estimates textual relevance from weighted term overlap rather than claiming full semantic understanding.

Pipeline: Raw text → cleaned text → term representation → TF-IDF matrix → vector comparison → similarity score → ranking.

12. TF-IDF Explained

TF-IDF means Term Frequency–Inverse Document Frequency. It assigns greater weight to terms that are important to a document while reducing the influence of terms that occur broadly across documents.

Conceptually: **TF-IDF(term, document) = TF(term, document) × IDF(term)**.

For recruitment, terms such as Python, SQL, Java, Machine Learning, TensorFlow or specific technologies can become useful signals when they distinguish relevant resumes from less relevant ones.

13. Cosine Similarity Explained

Cosine similarity compares two document vectors using the angle between them. It is widely used with TF-IDF because it focuses on vector direction rather than raw document length.

Conceptually: **cosine(A,B) = (A · B) / (||A|| × ||B||)**.

A higher similarity indicates greater alignment in weighted terms. The value is a relevance signal, not a probability that a candidate will perform successfully.

14. Candidate Scoring & Ranking

For every candidate, the system compares the processed resume vector against the job-description vector. The resulting similarity is used as the matching score, after which candidates can be sorted in descending order.

Illustrative example (not actual benchmark results):

Candidate	Similarity	Rank
Candidate A	0.86	1
Candidate B	0.73	2
Candidate C	0.61	3

Production thresholds should be validated against labeled resume/job pairs rather than selected arbitrarily.

15. Streamlit Application

Streamlit provides the browser-based presentation layer and integrates directly with Python ML/NLP functions.

Typical UI sections include job-description input, resume upload, matching controls, processing status, candidate scores and ranking results.

Benefits include rapid ML prototyping, simple deployment, and an interface that is easy to demonstrate during interviews or portfolio reviews.

16. Installation & Setup

Use a clean Python environment and keep scientific packages compatible. Compiled packages such as NumPy and Scikit-learn must be installed consistently within the same environment.

Example Windows setup:

```
cd C:\Users\HP\Downloads\resume_screener\resume_screener
python -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
pip install -r requirements.txt
```

If PowerShell blocks activation because of execution policy, Command Prompt can be used or the virtual environment's Python executable can be invoked directly.

17. How to Run

After installing dependencies, run:

```
python -m streamlit run app.py
```

Streamlit starts a local web server. Open the displayed local URL, provide the job description and resumes, and execute the matching workflow.

18. Testing Strategy

Unit testing: Test parsing, preprocessing, similarity, scoring and ranking independently.

Integration testing: Verify that the UI correctly communicates with backend functions.

Input testing: Test empty inputs, unsupported formats, very short text, long documents and duplicate resumes.

Algorithm testing: Identical/high-overlap text should generally score higher than unrelated text.

Regression testing: Maintain representative test resumes and verify ranking behavior after code changes.

19. Error Handling & Troubleshooting

A common issue in Python ML projects is NumPy/Scikit-learn binary incompatibility. An error such as **numpy.dtype size changed, may indicate binary incompatibility** usually indicates that compiled packages were built against incompatible NumPy interfaces.

A practical fix is to reinstall compatible NumPy and Scikit-learn versions together inside the same Python environment, verify the installed versions, and rerun the application. Avoid mixing packages from different Python installations.

Other common issues include missing dependencies, wrong working directories, unsupported file types, empty extracted text, and dependency-version conflicts.

20. Security & Privacy Considerations

Resumes can contain personal information. Avoid unnecessary storage and logging of raw resume contents; validate uploaded files and restrict file types/sizes; protect any stored data; use authentication and secure transport for public deployments; and never use an automated score as the sole employment decision.

21. Limitations

TF-IDF is primarily lexical and can miss semantic relationships when different words express the same idea.

Keyword frequency does not prove proficiency. Document extraction and formatting can affect scores. Similarity scores require validation and careful interpretation.

The current approach does not independently verify candidate claims and does not capture every human recruitment factor.

22. Future Enhancements

Semantic Embeddings: Add Sentence Transformers or another embedding model to identify related concepts even when exact keywords differ.

Skill Extraction: Use NLP/NER or an LLM to extract technical skills, soft skills, education, certifications, job titles and experience.

Hybrid Matching: Combine lexical similarity, semantic similarity, explicit skills, experience and education into a configurable score.

LLM-Based Analysis: Generate structured candidate summaries, matched requirements and missing requirements with appropriate privacy controls.

RAG: Retrieve approved job requirements, company policies or recruitment knowledge before generating grounded explanations.

Vector Database: Store candidate embeddings for fast semantic retrieval at larger scale.

Explainable Matching: Show matched skills, missing skills and supporting evidence behind a score.

Evaluation Framework: Create labeled resume/job pairs and measure ranking quality with Precision@K, Recall@K, MRR or NDCG.

23. Deployment Considerations

For portfolio demonstrations, Streamlit is appropriate. A production architecture could separate the UI, API, parsing service, ML inference, database and vector-search layer.

Production readiness should include pinned dependencies, environment variables for secrets/configuration, secure logging, automated tests, monitoring, authentication and reproducible deployment.

24. Conclusion

This project demonstrates how NLP and machine-learning fundamentals can be transformed into a practical end-to-end application. It combines document processing, text preprocessing, TF-IDF feature engineering, cosine similarity, candidate ranking, modular Python development and Streamlit presentation.

The system is also a strong baseline for a more advanced AI recruitment platform using semantic embeddings, transformer models, LLMs, RAG, vector databases and explainable AI.

25. Suggested GitHub README Content

- Project title and one-line description
- Problem statement and objectives
- Features and architecture
- Technology stack
- Project structure
- How the system works
- Installation and usage
- Algorithm explanation
- Screenshots and sample output
- Testing and troubleshooting
- Limitations and privacy considerations
- Future enhancements

- License and author information

Documentation note: This document reflects the project architecture and technologies described for the current Resume Screener. If your actual parser filenames, supported formats, score formula, or UI components differ, update those specific details in the GitHub README.